

03 — Data Plane

03 — Data Plane

Revision: 1 **Last modified:** 2026-07-05T14:20:00Z **Status:** Draft
— consolidated from 01-data-plane.md and v02-data-plane/*.

1. Position and invariants

The data plane is the **Rust code that moves already-encrypted bytes**. It is shared byte-for-byte by Client, Connector, and Gateway edge.

#	Invariant
I1	Transport never sees plaintext — it carries only encrypted WG datagrams.
I2	Transport carries unreliable datagrams, preserving WG loss semantics.
I3	Control plane is never in the packet path; fail-static when control is down.
I4	One transport crate, three consumers (client, connector, edge).
I5	No durable per-connection/per-packet state in the data plane.
I6	Default-deny: no peer reaches anything without an explicit policy rule.

2. Layering

L4 Application traffic
L3 WireGuard (Noise IK, Curve25519, ChaCha20-Poly1305) – fixed crypto core
L2.5 DAITA shaping (optional, above WG)
L2 Pluggable transport (one active):
 plain-udp · masque-h3 · shadowsocks · udp-over-tcp · lwo ·

hysteria2 (option)
L1 IP to gateway public endpoint

3. Crate layout

```
helix-core/
├── crates/
│   ├── helix-transport/  # Transport trait + all carriers
│   ├── helix-wg/         # boringtun wrapper
│   ├── helix-tun/        # OS TUN abstraction
│   ├── helix-daita/      # maybenot shaping stage
│   └── helix-route/      # overlay addressing + verdict-map
└── compiler
    ├── helix-core/       # orchestrator + status enum + ladder
    └── bin/
        ├── helix-client.rs
        ├── helix-connector.rs
        └── helix-edge.rs
```

4. The Transport trait

The single abstraction beneath WireGuard. Full binding signature lives in [../v02-data-plane/transport-trait.md](#).

```
#[async_trait]
pub trait Transport: Send + Sync {
    async fn send(&self, datagram: Bytes) -> Result<(),
        TransportError>;
    async fn recv(&self) -> Result<Bytes, TransportError>;
    fn kind(&self) -> &'static str;
    fn effective_mtu(&self) -> u16;
    fn health(&self) -> TransportHealth;
    async fn close(&self) -> Result<(), TransportError>;
}
```

TransportConfig enumerates carriers; dial(cfg) constructs the active transport.

5. Transport implementations

Transport	Phase	Notes
plain-udp	P0/P1	Baseline; effective_mtu() 1420; throughput gate G1 (≥80% bare link).
masque-h3	P0/P1	WG-over-HTTP/3 (RFC 9298/9297/9221) on :443/udp; the headline obfuscation.

Transport	Phase	Notes
lwo	P1	Lightweight keyed WG-header obfs + padding; first escalation rung.
shadowsocks	P2	WG-in-Shadowsocks AEAD stream for QUIC-hostile networks.
udp-over-tcp	P2	Last resort when UDP fully blocked; accepts head-of-line blocking.
connect-ip	P2	RFC 9484 IP-over-H3, advanced, feature-gated.
hysteria2	P1/P2	Optional ladder rung behind Cargo feature (decision D1 Camp B).

6. Auto-escalation ladder

Default order (unrestricted): [plain-udp]

Restricted/censored prior: [plain-udp, lwo, masque-h3, shadowsocks, udp-over-tcp]

The client walks `TransportPolicy.order` on repeated handshake failure; on success it remembers the working transport per-network (SSID / gateway fingerprint). The coordinator may push regional priors.

7. Overlay addressing and routing

- **Decision D4:** IPv6 ULA /48 per tenant + Tailscale-style **4via6** mapping of advertised IPv4 LANs.
- Every node (client, connector, advertised host) gets a stable overlay address in the tenant ULA /48.
- Connectors advertise served CIDRs; the control plane compiles a routing map pushed via `WatchNetworkMap`.
- Agents are declarative reconcilers: diff desired-vs-actual `RouteMap` and converge without restart.

8. Policy → AllowedIPs + verdict map

Default-deny, fail-closed:

1. **WireGuard AllowedIPs** — coarse crypto-enforced routing per peer.
2. **Edge verdict map** — port/proto-granular nftables (Phase 1) or eBPF (Phase 2) map.

Peers are delivered to each agent **already policy-filtered** (need-to-know).

9. Connector local-ACL × central policy precedence (GAP-1 closed in consolidation)

Adopted precedence rule (pending coordinator confirmation):

1. **Local-deny overrides central-allow** — a Connector may tighten policy for its own network.
2. **Central-deny overrides local-allow** — the tenant-wide default-deny/fail-closed invariant wins.
3. The compiled AllowedIPs + edge verdict map equal the **union of central policy minus local-deny**.
4. The Connector advertises its `local_denylist` to the coordinator so the edge enforces it consistently.

Honesty note: backported into `v03-control-plane/svc-policy.md` and `v04-client/helix-core-rust.md`; GAP-1 CLOSED.

10. DAITA, multi-hop, P2P

- **DAITA** (Phase 2): constant packet sizing + cover traffic via maybenot-style state machine, placed above WG.
- **Multi-hop** (Phase 2): nested WireGuard with per-hop keys; entry sees client but not destination, exit sees destination but not client.
- **P2P + NAT traversal** (Phase 2): STUN-like discovery, hole punching, DERP-style helix-relay fallback; WatchNetworkMap is the signaling channel.

11. MTU budget

Transport	effective_mtu()
plain-udp	1420
masque-h3	1280 (measure & tune)
shadowsocks	~1380
udp-over-tcp	~1380
lwo	~1400

12. Cross-references

- Binding Transport trait → [../v02-data-plane/transport-trait.md](#)
 - Per-transport deep-dives → [../v02-data-plane/](#)
 - Control-plane policy compiler → [04 — Control Plane §6](#)
 - Client core packaging → [05 — Client Core & UI](#)
-

Sources: docs/research/mvp/final/01-data-plane.md, v02-data-plane/transport-.md, v02-data-plane/routing-and-addressing.md, v02-data-plane/multihop.md, v02-data-plane/daita.md, v02-data-plane/orchestrator-and-state.md.*